

KIỂM TRA THỰC HÀNH MÔN JAVA WEB SERVICE SS13
THỰC HÀNH AOP, LOGGING, UNIT TESTING - SESSION 13
MÔN HỌC JAVA WEB SERVICE
THỜI GIAN: 45 phút

Yêu cầu:

- Tạo github repository theo cú pháp : *[Tên lớp]_[Họ Tên]_[Kiểm Tra Thực Hành]*
- Ví dụ: *HCM-K24-CNTT2_NguyenVanA_KiemTraThucHanhSession13*
- Sau khi hoàn thành, đẩy code lên github repo và nộp link cho người phụ trách
- Công nghệ sử dụng: **Java**
- IDE : **IntelliJ**, MySQL Workbench, Postman
- *Lưu ý tuyệt đối không sử dụng Chat-GPT hay AI để làm bài, không copy bài người khác, nếu bị phát hiện sẽ lập biên bản và xử lý theo quy định*

I. Cấu hình môi trường

Tạo project Spring Boot (Maven hoặc Gradle) với các dependency:

- Spring Web
- Spring Data JPA
- MySQL Driver
- Spring Boot DevTools
- Spring AOP
- Lombok

Yêu cầu kỹ thuật

- Sử dụng MySQL để lưu dữ liệu
- Sử dụng Spring Data JPA thao tác database
- Trả dữ liệu bằng ResponseEntity

II. Yêu cầu thực hiện

1. Tạo entity Book

Tạo class Book trong package entity với domain: **Hệ thống Quản lý Thư Viện Sách** gồm các thuộc tính:

Thuộc tính	Kiểu dữ liệu
id	Long

title	String
author	String
category	String
quantity	Integer

Yêu cầu

- Sử dụng annotation JPA để mapping database
- id tự tăng (AUTO_INCREMENT)
- Sử dụng Lombok (@Data, @NoArgsConstructor, @AllArgsConstructor)

2. Tạo BookRepository

Tạo interface **BookRepository** trong package **repository**.

3. Tạo BookService và BookController theo chuẩn REST

Tạo **BookService** trong package **service** và **BookController** trong package **controller**.

Triển khai các API sau:

Method	Endpoint	Mô tả	Thành công	Lỗi
GET	/api/books	Lấy danh sách sách	200 OK	-
GET	/api/books/{id}	Lấy sách theo ID	200 OK	404 Not Found
POST	/api/books	Thêm sách mới	201 Created	400 Bad Request
PUT	/api/books/{id}	Cập nhật toàn bộ	200 OK	404 Not Found
PATCH	/api/books/{id}	Cập nhật một phần	200 OK	404 Not Found
DELETE	/api/books/{id}	Xóa sách	204 No Content	404 Not Found

III. Yêu cầu AOP (Aspect-Oriented Programming)

1. Tạo BookServiceAspect

Tạo class **BookServiceAspect** trong package **aspect**, áp dụng pointcut cho toàn bộ phương thức trong package **service**:

- **@Before**: In log tên phương thức và tham số đầu vào trước khi method thực thi
- **@AfterReturning**: In log kết quả trả về sau khi method thực thi thành công
- **@AfterThrowing**: In log thông tin lỗi khi method ném ra ngoại lệ

2. Custom Exception và GlobalExceptionHandler

- Tạo class **BookNotFoundException** (extends **RuntimeException**) — ném khi không tìm thấy sách theo ID
- Tạo **GlobalExceptionHandler** dùng **@RestControllerAdvice** để bắt **BookNotFoundException**, trả về HTTP 404 với message phù hợp

Gợi ý pointcut: `execution(* com.example.library.service.*.*(..))`

IV. Yêu cầu Logging

1. Cấu hình Logback

Tạo file **logback-spring.xml** trong **src/main/resources** với:

- Console Appender: In log ra terminal, pattern gồm: [thời gian] [level] [tên logger] - message
- File Appender: Ghi vào file **logs/library-app.log**, áp dụng **RollingPolicy** theo ngày, giữ tối đa 7 file
- Root level: **INFO** — package **com.example.library**: **DEBUG**

2. Log trong BookService

Dùng **@Slf4j** (Lombok) để thêm log ở các mức:

- **DEBUG**: Log tham số đầu vào khi nhận request
- **INFO**: Log kết quả khi thực hiện thành công
- **ERROR**: Log message lỗi khi xảy ra ngoại lệ

V. Yêu cầu Unit Test

Sử dụng **JUnit 5 + Mockito**. Tạo test trong **src/test/java**.

1. BookServiceTest

Viết test cho các method của **BookService**:

Tên test method	Kịch bản cần kiểm tra
-----------------	-----------------------

getAllBooks_returnList	Mock repository trả về 2 cuốn sách → Service trả về đúng 2 phần tử
getBookById_found	Mock repository findById trả về Optional có giá trị → Service trả về đúng Book
getBookById_notFound	Mock repository findById trả về Optional.empty() → Service ném BookNotFoundException
createBook_success	Mock repository save → Service trả về Book vừa tạo, gọi save đúng 1 lần
deleteBook_notFound	Mock findById trả về Optional.empty() → Service ném BookNotFoundException, không gọi delete

2. BookControllerTest

Dùng **@WebMvcTest** + **MockMvc** để viết test cho:

- GET /api/books → trả về 200 OK và danh sách JSON
- GET /api/books/{id} khi tìm thấy → trả về 200 OK và object JSON
- GET /api/books/{id} khi không tìm thấy → trả về 404 Not Found
- POST /api/books → trả về 201 Created và object vừa tạo

VI. Kiểm thử với Postman

Dùng Postman kiểm thử đầy đủ các trường hợp sau:

STT	Nội dung
1	GET all – thành công (200)
2	GET by id – thành công (200)
3	GET by id – không tồn tại (404)
4	POST – thêm mới thành công (201)
5	PUT – cập nhật toàn bộ thành công (200)
6	PUT – ID không tồn tại (404)
7	PATCH – chỉ cập nhật quantity (200)

8	PATCH – ID không tồn tại (404)
9	DELETE – xóa thành công (204)
10	DELETE – ID không tồn tại (404)

VII. Hình thức nộp bài

Sinh viên nộp:

- Source code project:
 - Nộp bài lên GitHub và dán link vào Portal
- Screenshot kết quả Run All Tests (tất cả test PASS)
- Postman Collection (.json)
 - Hoặc screenshot kết quả test API